Control Unit

Control Unit, ID (Instruction Decode) aşamasında aktif olan kritik bir bileşendir. Bu ünitenin temel görevi, gelen komutun opcode ve function code alanlarına göre kontrol sinyallerini üretmektir. Logisim tasarımımızda bu amaçla iki adet ROM (Read Only Memory) modülü kullanılmıştır.

Kontrol Ünitesi iki adet giriş alır:

- **Opcode** (komut türünü belirler)
- **Function Code** (R-type komutlarda kullanılan işlem türü)

ROM Yapısı ve Seçim Mekanizması

ROM'ların biri opcode temelli komutlar için, diğeri ise function code'a göre çalışan R-type komutlar için yapılandırılmıştır. Bu iki ROM'dan hangisinin aktif olacağı bir **MUX** (Multiplexer) ile belirlenir. Bu MUX'un seçim sinyali, opcode'un 000 (yani R-type komut) olup olmamasına göre belirlenir. Bu kontrol, opcode'un her bir bitini sıfırla karşılaştıran ve sonucunda bir AND kapısından çıkan sinyal ile yapılır. Yani:

- opcode = 000 olduğunda AND kapısından 1 çıkar ve Function Code ROM'u aktif olur.
- opcode ≠ 000 olduğunda diğer ROM (Opcode ROM) seçilir.

Her iki ROM'un içeriği manuel olarak girilmiştir. Girilen hex kodları ve karşılık geldikleri komut türleri, ekteki Excel dosyasında detaylı şekilde belirtilmiştir

Örnek:

- Function Code = 100000 → ADD komutu
- Function Code = 100010 → SUB komutu
- Function Code = 100100 → AND komutu

12 Bitlik Kontrol Sinyali

ROM çıkışında toplam **12 bitlik** bir kontrol sinyali üretilmektedir. Bu 12 bitlik veri, ilgili işlem için gerekli olan tüm kontrol sinyallerini tek seferde taşır.

Bu 12 bitin anlamı şu şekildedir (soldan sağa doğru):

1. **[0-3] ALUControl** – ALU'nun hangi işlemi yapacağı

2. **[4] ALUSrc** – İkinci ALU operandının kaynaktan mı immediate'tan mı geleceği
3. **[5] MemWrite** – Belleğe yazma sinyali
4. **[6] MemToReg** – Bellekten gelen verinin register'a yazılıp yazılmayacağı
5. **[7] RegWrite** – Register dosyasına yazma izni
6. **[8] RegDst** – Hangi register'a yazılacağını seçer (rd mi rt mi)
7. **[9] Branch** – Dallanma olup olmadığı
8. **[10] MemRead** – Bellekten okuma izni
9. **[11] Jump** – Zıplama işlemi olup olmadığı

Her bir komut için bu 12 bitlik kontrol sinyali hesaplanmış ve ilgili ROM içerisine girilmiştir.

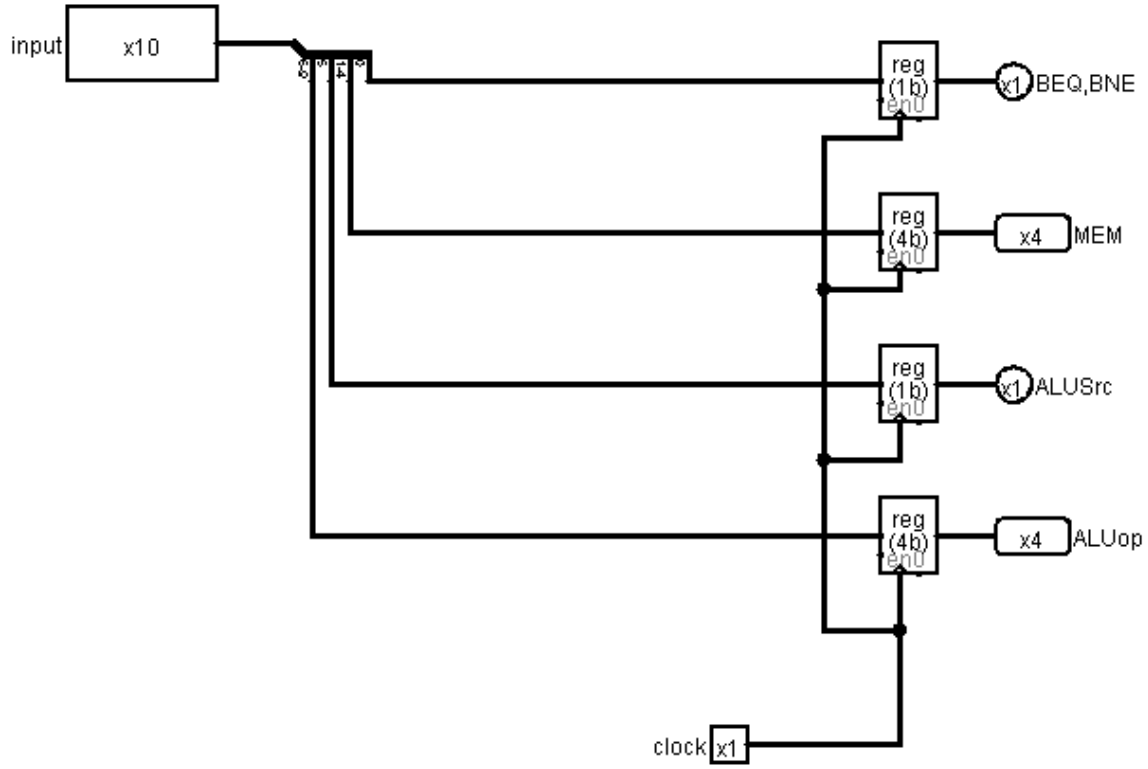
2.3. Sinyal Ayırımı ve İletimi

ROM çıkışından alınan 12 bitlik kontrol sinyali bir **splitter** aracılığıyla ayrılmıştır. ID aşamasında bu sinyallerin yalnızca iki tanesi aktif olarak ilgili yerlere yönlendirilir:

- **RegDst**
- **Jump**

Bu iki sinyal, ID aşamasından direkt olarak hedef devrelere yönlendirilirken, kalan 10 bitlik sinyaller sonraki EX aşamasına taşınmak üzere bir **Control Register (ID/EX Register)** içerisine aktarılır. Böylece kontrol sinyalleri, sonraki aşamalarda da komutun doğru bir şekilde işlenmesine olanak tanır.

EX (Execute) Aşaması



EX aşamasına, Control Unit tarafından üretilmiş ve bir önceki pipeline register'ı üzerinden taşınmış toplam **10 bitlik bir kontrol sinyali** girişi yapılmaktadır. Bu 10 bitlik verinin yapısı, daha önce oluşturduğumuz "**Control Unit Bit Sıralaması**" isimli Excel tablosuna göre belirlenmiştir. Bu tabloda, Jump ve RegDst bitleri EX aşamasından önceki ID aşamasında kullanıldığından, EX aşamasına sadece geri kalan 10 bit aktarılmaktadır.

Bu 10 bitlik sinyal bir **splitter** ile ayrıştırılmaktadır. Bitlerin anlamları şu şekilde sıralanmıştır:

- **Bit 0 (en düşük değerli bit)**: BranchNotEqual (BNE) sinyali
- **Bit 1–4 (4 bit)**: Memory Control sinyalleri (örneğin MemRead, MemWrite gibi)
- **Bit 5**: ALUSrc sinyali
- **Bit 6–9 (4 bit)**: ALUOp sinyalleri (ALU işleminin türünü belirler)

Bu şekilde ayrıştırılan sinyallerin kullanımı şu şekildedir:

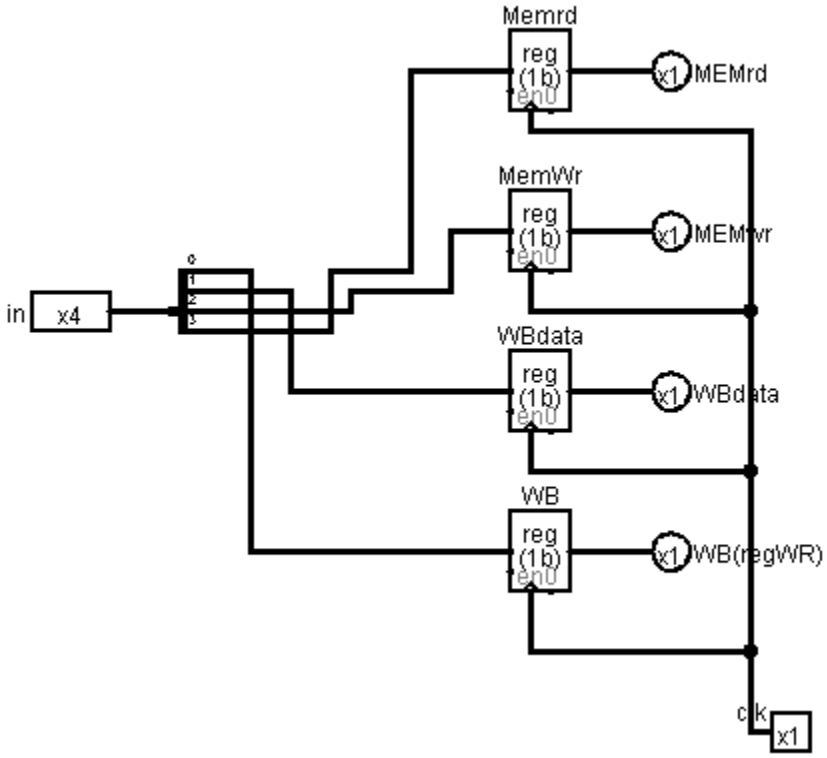
- **ALUSrc biti**, doğrudan bu aşamadaki **MUX** bileşenine bağlanarak, ALU'ya gönderilecek operand'ın seçilmesinde etki eder.

- **ALUOp (4 bit)** ve **Memory Control (4 bit)** sinyalleri, doğrudan kullanılmaz; bir sonraki aşama olan MEM (Memory) aşamasına aktarılmak üzere **EX/MEM register'ına** bağlanır.

- **BranchNotEqual (BNE)** sinyali ise, bu aşamada **PC Control** birimiyle ilişkilendirilir. Bu sinyalin aktif olması durumunda, belirli koşullara göre program sayacının (PC) değiştirilmesi sağlanır.

Her bir çıkış sinyali, uygun clock sinyali eşliğinde ilgili register'lara aktarılmak üzere bağlanmıştır. Bu sayede, pipeline yapısının senkronizasyonu sağlanmakta ve işlem adımları arasında tutarlılık korunmaktadır.

MEM (Memory Access) Aşaması



MEM aşaması, EX aşamasından gelen kontrol sinyilerini ve ALU sonuçlarını işleyen, veri belleği (RAM) ile etkileşim kuran aşamadır. Bu aşamada, EX/MEM pipeline register'ı üzerinden **4 bitlik bir kontrol sinyali** MEM aşamasına aktarılır. Bu 4 bitlik sinyalin anlamları ve bağlantıları şu şekildedir:

- **Bit 0 (en düşük değerlikli bit)**: RegWrite (WB) sinyali
    - Bu sinyal, **bir sonraki aşama olan WB (Write Back)** aşamasında register dosyasına veri yazılıp yazılmayacağını belirler. WB aşamasına aktarılır.
- **Bit 1**: WBData sinyali
    - Bu sinyal, **RAM'den gelen veri** ile **ALU sonucu** arasında seçim yapan bir **MUX**'un kontrol bitidir. MUX'un çıkışı, WB aşamasında register dosyasına yazılacak veriyi belirler. WB aşamasına aktarılır.
- **Bit 2**: MemRead sinyali
    - Bu bit, **veri belleğinden okuma işlemi** yapılıp yapılmayacağını belirler. Aktifse, ALU sonucunda hesaplanan adres kullanılarak RAM'den veri okunur.
- **Bit 3**: MemWrite sinyali
    - Bu bit, **veri belleğine yazma işlemi** yapılacaksa aktif hale gelir. Yazılacak veri, EX aşamasından gelen register içeriğidir.

Bu kontrol sinyalleri, ilgili bileşenlere doğrudan bağlanır:

- MemRead ve MemWrite sinyalleri, **RAM bileşenine** doğrudan bağlanarak okuma/yazma işlemlerini yönetir.
- RegWrite ve WBData sinyalleri, **MEM/WB register'ına** bağlanarak bir sonraki aşama olan WB aşamasına aktarılır.

Bu yapıda dikkat edilmesi gereken bir nokta, **ALU Operation sinyallerinin** aslında EX aşamasında kullanıldığıdır. Bu sinyaller, ALU'ya hangi işlemi yapacağını belirtir (örneğin AND, OR gibi). Önceki açıklamada yanlışlıkla bu sinyallerin MEM aşamasına aktarıldığı belirtilmişti, ancak gerçekte **ALUOp sinyalleri sadece EX aşamasında kullanılmaktadır.**

Sonuç olarak, MEM aşamasında sadece **4 bitlik kontrol sinyali** işlenmekte, bunlardan yalnızca 2 tanesi bir sonraki WB aşamasına aktarılmaktadır. Diğer 2 bit ise RAM ile doğrudan etkileşim kurmaktadır.

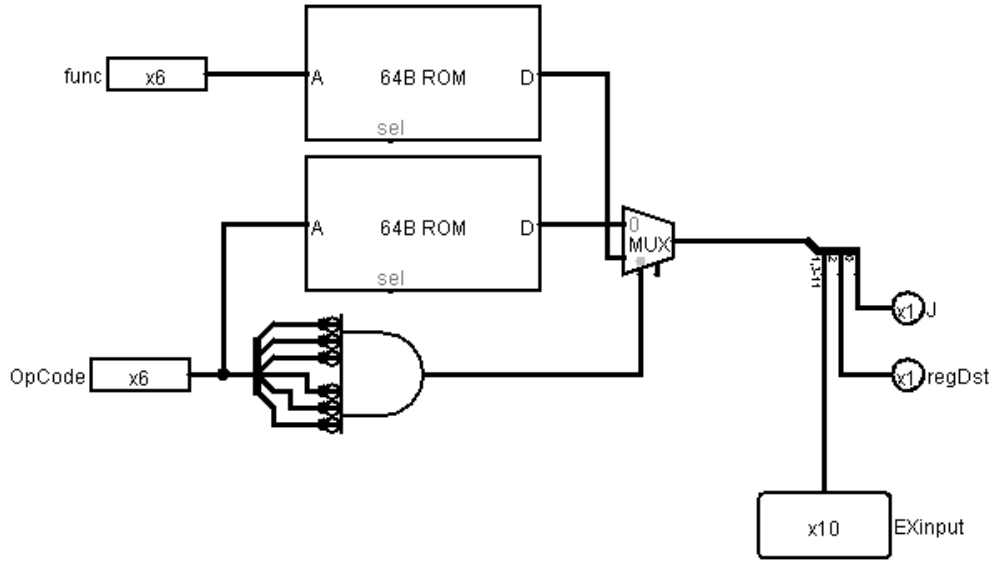
MEM/WB Register (WB Aşaması)

MEM/WB register'ı, **MEM (Memory)** aşamasından gelen sinyalleri alarak **son aşama olan Write Back (WB)** aşamasına aktarır. Bu register, sistemin doğru çalışabilmesi açısından oldukça kritiktir çünkü CPU'nun register dosyasına veri yazıp yazmayacağı bu aşamada belirlenir.

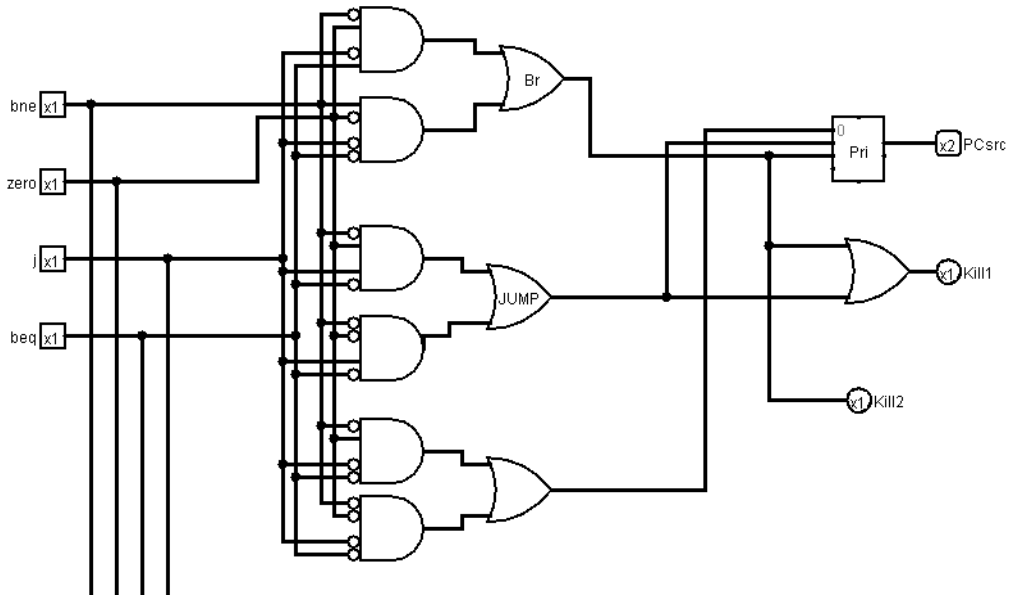
MEM aşamasından gelen ve MEM/WB register'ında tutulan sinyaller:

- **RegWrite (1 bit)**
  - Bu sinyal, hedef register'a yazma işleminin yapılıp yapılmayacağını belirler.
    RegWrite = 1 ise: Yazma işlemi gerçekleşir.
    RegWrite = 0 ise: Yazma işlemi yapılmaz.
  - Bu kontrol sinyali, **register file bloğuna** bağlanır ve yazma izin sinyali olarak kullanılır.
- **WBData (1 bit)**
  - Bu sinyal, WB aşamasındaki **veri kaynağını belirler.**
    Sinyale bağlı olan MUX, iki girişten (RAM'den okunan veri veya ALU sonucu) birini seçer. Seçilen veri, hedef register'a yazılır.

WB aşamasında herhangi bir hesaplama yapılmaz, sadece **kontrol sinyallerine göre** gerekli veri hedef register'a yazılır. Eğer `RegWrite` sinyali aktif değilse (0 ise), hiçbir yazma işlemi gerçekleştirilmez.



PC Control Birimi



PC Control birimi, **Program Counter (PC)**'ın bir sonraki adımda ne yapacağına karar veren kontrol mekanizmasıdır. Bu birim, sistemin sıçrama (jump) veya dallanma (branch) yapıp yapmayacağını belirlemek için birkaç giriş sinyali alır ve uygun çıkışı üretir.

Girişler

1. **Branch Not Equal (BNE)**
   - EX aşamasındaki **EX Register** üzerinden gelir.
   - Eğer aktifse ve `Zero = 0` ise, PC yeni adrese yönlendirilir (eşit değilse dallan).
2. **Branch If Equal (BEQ)**
   - EX Register üzerinden gelir.
   - Eğer aktifse ve `Zero = 1` ise, PC dallanır (eşitse dallan).
3. **Zero**
   - ALU'nun `Zero` çıkışından gelir.
   - İki operand eşitse 1, eşit değilse 0 olur.
4. **Jump (J)**

- **Control Unit**'ten doğrudan gelen bir sinyaldir.
- Eğer aktifse, PC sıçrama adresine yönlendirilir.

İç Yapı ve Mantık Kapıları

PC Control birimi içinde bir dizi **AND kapısı** ve bir **Encoder** yer alır. Bu kapılar, çeşitli sıçrama ve dallanma koşullarını kontrol eder:

1. **Üstteki AND kapıları (Branch kontrolü):**
   - `Branch If Equal` ve `Zero = 1` ⇒ BEQ başarılı ⇒ Branch yapılabilir.
   - `Branch Not Equal` ve `Zero = 0` ⇒ BNE başarılı ⇒ Branch yapılabilir.
2. **Jump kontrolü yapan AND kapıları:**
   - Jump sinyali aktifken `Zero = 1` veya `Zero = 0` olabilir. İki durumda da Jump yapılabilir.
   - Bu yüzden iki ayrı AND kapısı kullanılır.
3. **Varsayılan yol (ne Jump ne de Branch yapılmıyorsa):**
   - Alt iki AND kapısı bu durumu kontrol eder.
   - **Jump = 0**, **Branch sinyalleri = 0** olduğunda bu kapılar aktif olur.

Encoder ve PCsrc Çıkışı

Tüm AND kapılarının çıkışları bir **2-bit Encoder**'a bağlanır. Encoder, aktif olan girişe göre bir çıkış kodu üretir:

- **00**: Normal ilerleme (`PC + 4`)
- **01**: Jump yapılacak
- **10**: Branch yapılacak
- **11**: Kullanılmayan (hiçbir sinyal bağlanmamış)

Bu çıkış, **PCsrc** sinyali olarak **Program Counter'ın başındaki 3 girişli MUX'a** gider:

- **Giriş 0**: `PC + 4` (normal artış)
- **Giriş 1**: Jump adresi
- **Giriş 2**: Branch adresi

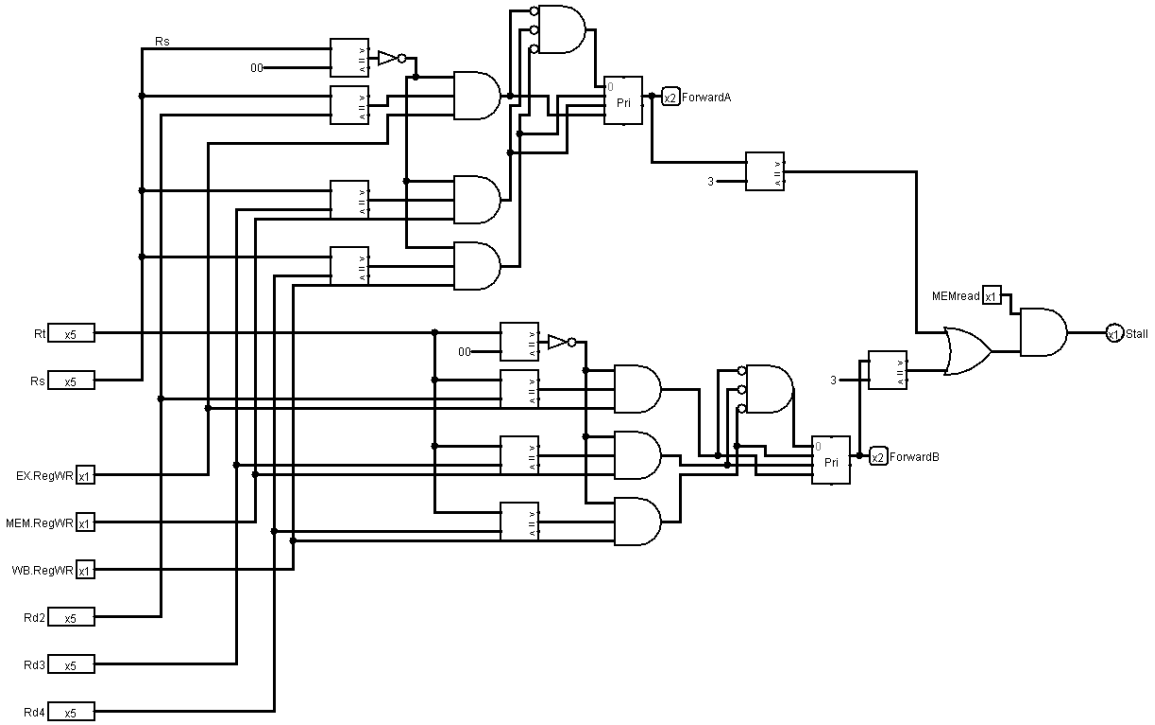
Kill Sinyalleri

PC Control birimi ayrıca **pipeline kontrolü** için iki adet kill sinyali üretir:

- **Kill 1**
    - Jump **veya** Branch yapılıyorsa aktif olur.
    - **Instruction Memory**'den çıkan komutların yazıldığı MUX'a bağlanır.
    - Gerektiğinde komutun stall edilmesini sağlar.
- **Kill 2**
    - Sadece **Branch** yapılacaksa aktif olur.
    - Control sinyallerinin geçtiği başka bir MUX'a bağlanır.
    - İlgili aşamada **bubble** yerleştirilmesini sağlar.

PC Control birimi, Branch ve Jump sinyallerine göre Program Counter'ın hangi adrese yönleneceğine karar verir. İçerisinde yer alan AND kapıları ve encoder sayesinde mantıksal karar mekanizması oluşturulmuştur. Aynı zamanda, pipeline kontrolü için gerekli olan kill sinyalleri ile uyumlu çalışır.

Hazard Detection Unit



Hazard Detection Unit'in temel amacı, veri çakışmalarını (data hazards) tespit etmek ve gerekli yönlendirmeleri (forwarding) ya da duraksamaları (stall) uygulamaktır. Bu birimde, RS ve RT olmak üzere iki adet kaynak (source) register girişi bulunur. Bu girişler üzerinden yapılan okuma işlemleri sırasında, daha önceki komutların aynı register'lara yazma yapma ihtimali değerlendirilerek çakışma olup olmadığı kontrol edilir. Eğer önceki komutlardan biri, henüz yazma işlemini tamamlamadan aynı register'a yazmak üzereyse, okuma işleminin beklemesi gerekir. Aksi takdirde, okunacak veri henüz güncellenmemiş olabilir.

Hazard kontrolünde, üç önceki komuta kadar olan yazma işlemleri göz önünde bulundurulur. Bu bağlamda, sırasıyla RD2 (bir önceki komut), RD3 (iki önceki komut) ve RD4 (üç önceki komut) register'ları üzerinden karşılaştırmalar yapılır. RD2, EX aşamasındaki komutun hedef register'ını; RD3, MEM aşamasındaki; RD4 ise WB aşamasındaki hedef register'ı temsil eder. Bu aşamaların her biri, kendilerine ait `RegWr` (Register Write) kontrol sinyali ile birlikte değerlendirilir.

Karşılaştırmalar iki temel doğrultuda yapılır:

1. **RS Register'ı ile karşılaştırmalar:**
    - RS = RD2 && EX.RegWr = 1 → Çakışma vardır.
    - RS = RD3 && MEM.RegWr = 1 → Çakışma vardır.
    - RS = RD4 && WB.RegWr = 1 → Çakışma vardır.
2. **RT Register'ı ile karşılaştırmalar:**
    - RT = RD2 && EX.RegWr = 1 → Çakışma vardır.
    - RT = RD3 && MEM.RegWr = 1 → Çakışma vardır.
    - RT = RD4 && WB.RegWr = 1 → Çakışma vardır.

Elde edilen karşılaştırma sonuçlarına göre, forwarding işlemi için bir encoder devresi kullanılır. Bu encoder, aşağıdaki gibi dört farklı giriş üzerinden çalışır:

- **Input 3 (11)**: Eğer RD2 ile eşleşme varsa (en güncel çakışma).
- **Input 2 (10)**: Eğer RD3 ile eşleşme varsa.
- **Input 1 (01)**: Eğer RD4 ile eşleşme varsa.
- **Input 0 (00)**: Eşleşme yoksa.

Bu yapı, hem RS hem de RT register'ları için ayrı ayrı kontrol edilir. Forwarding kararları, `forwardA` ve `forwardB` sinyalleri aracılığıyla yönlendirme mantığına aktarılır. Bu sinyallerin değerlerine göre ilgili MUX'lar aşağıdaki veri kaynaklarını seçer:

- **00**: Register dosyasındaki veri (normal okuma).
- **01**: WB aşamasından gelen yazılacak veri (BusW).
- **10**: MEM aşamasından gelen veri.
- **11**: EX aşamasındaki ALU çıktısı.

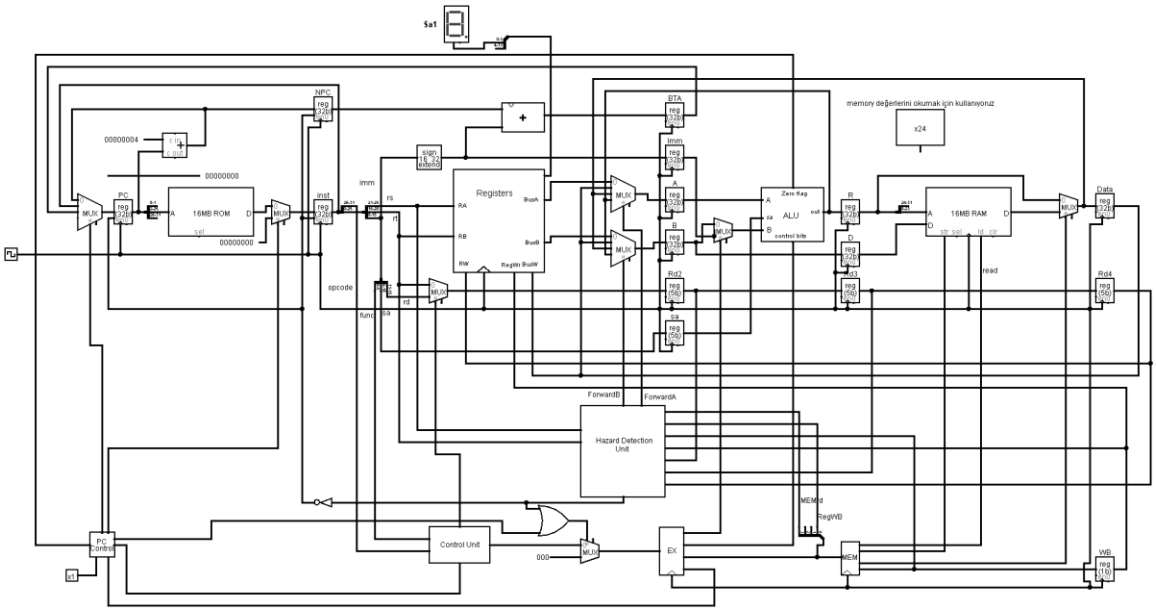
Bu sayede, çakışma durumunda veri eksikliği önlenmiş olur.

Stall Mekanizması

Hazard Detection Unit'in bir diğer kritik işlevi de `stall` sinyali üretmektir. Bazı durumlarda forwarding yeterli olmaz ve işlem hattında duraklama (bubble) oluşturmak gerekir. Stall sinyali aşağıdaki koşullarda aktif hale gelir:

- Eğer `forwardA` ya da `forwardB` sinyali 3 (yani 11) değerini alıyorsa (yani çakışma EX aşamasındaki komutla ise),
- Aynı zamanda EX aşamasındaki komutun `RegWr` sinyali 1 ise (yani yazma yapacaksa),
- Ve şu anki komut `MemRead` sinyali 1 ise (yani bellekten veri okumaya çalışıyorsa),

Bu durumda, işlem hattında bir adet duraklama sinyali (`stall = 1`) üretilir ve pipeline'da bir adet bubble yerleştirilir. Böylece, veri tutarsızlıkları engellenmiş olur.



Instruction Fetch (IF) Aşaması

CPU tasarımımızın ilk aşaması olan *Instruction Fetch (IF)* kısmında, komutların bellekteki adreslerinden okunarak işlenmeye hazır hale getirilmesi sağlanır. Bu aşamada kullanılan temel bileşenler: birden fazla MUX, Program Counter (PC), Instruction Memory (ROM), Instruction Register ve bazı kontrol birimleridir.

İlk olarak, sistem bir MUX ile başlar. Bu MUX'un üç farklı girişi ve iki bitlik bir seçme hattı bulunmaktadır. Girişlerden ilki, bir sonraki komut adresini (PC + 4) temsil eder ve normal çalışmada varsayılan olarak seçilen yoldur. İkinci giriş, eğer *jump* komutu varsa, atlama adresini sağlar. Üçüncü giriş ise *branch* komutu durumunda kullanılır. Bu MUX'un seçme sinyali, "PC Control" adını verdiğimiz ve *Control Unit* tarafından yönetilen birimden gelir. Bu sayede, hangi tür komutun işlendiğine göre uygun adres seçilir.

MUX'un çıkışı Program Counter (PC) register'ına bağlıdır. PC, çalıştırılacak komutun adresini tutar ve her saat darbesinde 4'er 4'er artar. Bu, her komutun 32 bit (4 byte) uzunlukta olması nedeniyle gereklidir. Ancak, Logisim'de kullandığımız hazır ROM modülü adresleri birer birer artıracak şekilde yapılandırılmıştır. Bu nedenle, adresin ilk iki biti her zaman sıfır olacağından, sadece kalan 24 biti kullanılır. Bu işlem, bir splitter aracılığıyla gerçekleştirilir. ROM'un 24 bitlik adres girişi olması nedeniyle bu çözüm tercih edilmiştir.

Instruction Memory olarak hazır bir ROM bileşeni kullanılmıştır. Bu belleğe komutlar hexadecimal formatta manuel olarak yüklenmektedir. ROM'un çıkışı, bir başka MUX'a bağlanmıştır. Bu MUX'un görevi, bazı özel durumlarda (örneğin branch nedeniyle gerçekleşen atlamalarda) "bubble" (boş komut) üretmektir. Bu amaçla, MUX'un bir girişi sıfıra bağlanmıştır. Diğer girişi ise instruction memory çıkışıdır. MUX'un seçme hattı yine PC Control birimi tarafından kontrol edilir.

Bu MUX'un çıkışı ise Instruction Register'a bağlanmıştır. Instruction Register, bellekteki komutun binary ya da hexadecimal karşılığını geçici olarak tutar. Böylece opcode, rs, rt gibi alanlar bu register üzerinden erişilebilir hale gelir.

Ayrıca, IF aşamasında bir adder bloğu kullanılarak mevcut PC değeri 4 ile toplanır ve elde edilen değer "Next PC" (NPC) olarak adlandırılan register'a yazılır. Böylece bir sonraki komutun adresi belirlenmiş olur.

Bu aşamada iki adet önemli kontrol sinyali bulunmaktadır:

1. **PC Başındaki MUX'un kontrolü:** Bu MUX'un seçme hatları "PC Control" biriminden gelen iki bitlik sinyal ile kontrol edilir. Komutun türüne göre doğru adres girişinin seçilmesi sağlanır.

2. **Bubble için kullanılan ikinci MUX'un kontrolü:** Bu MUX'un kontrolü ise "kill1" adı verilen bir sinyal ile yapılır. Branch gibi durumlarda geçici olarak "boş komut" gönderilmesini sağlar.

Son olarak, bu aşamada yer alan üç register (PC, Instruction Register ve NPC) ortak bir kontrol sinyali ile etkinleştirilir. Bu sinyal, "Hazard Detection Unit" tarafından üretilen "stall" sinyalidir. Stall sinyali aktif olduğunda, bu register'ların enable girişleri devreye girer ve bir sonraki komut alınmadan önce sistem geçici olarak durdurulabilir. Bu, veri çatışmalarını engellemek için kritik bir mekanizmadır.

Instruction Decode (ID) Aşaması

Instruction Decode (ID) aşaması, komutun çözümlenerek işlemeye hazır hale getirildiği ve yürütmeye hazırlıkların yapıldığı kritik bir aşamadır. Bu aşamada temel olarak üç ana yapı kullanılır: Instruction Register, Register File ve A/B register'ları. IF aşamasından gelen 32-bit'lik komut, ilk olarak Instruction Register'a yazılır. Ardından bu komut, opcode, function, rs, rt, rd, shift amount ve immediate alanlarına bölünür. Opcode ve function bilgileri, komutun türünü çözümlemek üzere Control Unit'e gönderilirken, rs ve rt adresleri Register File'ın girişlerine bağlanır. rd alanı ise, destination register belirlemede kullanılır. Bu alanlar, instruction'ın yapısına göre bir multiplexer (MUX) ile seçilir. Eğer komut rt'ye yazacaksa rt, rd'ye yazacaksa rt seçilir; bu MUX'un kontrolü Control Unit tarafından sağlanır. Shift amount (SA) alanı ise, EX aşamasında kullanılmak üzere SA adlı özel bir register'a aktarılır. Immediate değeri 16-bit olduğu için, Sign Extender yardımıyla 32-bit'e çıkarılarak Immediate Register'a yazılır.

Register File içerisinden okunan iki veri, BUS A ve BUS B hatlarına gönderilir. Ancak doğrudan A ve B register'larına bağlanmak yerine, veri çakışmalarına (hazard) karşı korunmak için iki ayrı MUX üzerinden geçerler. Bu MUX'ların kontrolü, Hazard Detection Unit tarafından yapılır. Eğer bir çakışma tespit edilirse, önceki aşamalardaki

sonuçlardan (EX, MEM ya da WB) uygun olanı MUX'a yönlendirilir. Böylece A ve B register'ları doğru verilerle güncellenmiş olur. Bu aşamanın bir diğer önemli görevi ise, branch komutları için olası atlama adreslerini hesaplamaktır. Immediate değer ile Next PC değeri bir adder bloğunda toplanarak, Branch Target Address belirlenir ve bu adres beta adlı bir register'a yazılır.

Control Unit, opcode ve function bilgilerine göre komutun türünü belirler ve buna uygun kontrol sinyalleri üretir (örneğin RegWrite, ALUSrc, MemRead, MemWrite gibi). Bu sinyaller, yalnızca ID aşamasında değil, sonraki EX, MEM ve WB aşamalarında da kullanılacağından, özel olarak oluşturulmuş EXReg, MEMReg ve WBReg register'larına yazılarak bir sonraki aşamalara taşınır. Hazard Detection Unit, yalnızca forwarding işlemleriyle sınırlı kalmaz, aynı zamanda üst üste gelen komutlar arasında bir çakışma varsa sistemin durmasını (stall) sağlar. Stall sinyali sayesinde PC ve IF/ID register'ları geçici olarak dondurulur ve kontrol sinyalleri sıfırlanarak "bubble" (NOP) yerleştirilir.

Execute (EX) Aşaması

Execute (EX) aşaması, komutun türüne göre aritmetik ya da mantıksal işlemlerin gerçekleştirildiği işlemci içi yürütme aşamasıdır. Bu aşamada başlıca kullanılan bileşenler şunlardır: **ALU (Arithmetic Logic Unit)**, **ALU Control**, **MUX'lar**, **Adder blokları** ve çeşitli **register'lar**.

Instruction Decode (ID) aşamasında elde edilen operandlar (veri girişleri), bu aşamada işlem görmek üzere kullanılır. ALU'ya gidecek girişlerin seçimi, bir multiplexer (MUX) yardımıyla yapılır. ALU'nun birinci girişi genellikle A register'ından gelirken, ikinci girişi bir MUX aracılığıyla belirlenir: bu giriş ya B register'ından gelen veri ya da Immediate değer olabilir. Bu seçim, ALUSrc kontrol sinyaliyle yapılır. Immediate değerin EX aşamasında kullanılması gereken durumlarda, Sign Extender tarafından genişletilmiş hali kullanılır.

ALU'nun hangi işlemi gerçekleştireceği, ID aşamasında opcode ve funct alanlarına bakılarak oluşturulan kontrol sinyallerine göre belirlenir. ALU Control birimi, opcode ve funct bilgisini yorumlayarak ALU için uygun işlem kodunu üretir. Bu sayede toplama, çıkarma, AND, OR, shift gibi işlemler doğru şekilde yürütülür.

EX aşamasında ayrıca **branch** (dallanma) işlemleri için gerekli adres hesaplamaları yapılır. Eğer komut bir dallanma komutuysa, PC + 4 değeri ile Immediate değeri (sign-extended) bir adder bloğunda toplanarak **Branch Target Address** hesaplanır. Bu hesaplanan adres, ileride Program Counter (PC) tarafından kullanılmak üzere Branch Target Register'a yazılır. Bu değer, branch komutunun gerçekleştirilip gerçekleştirilmemesi durumuna göre ilerleyen aşamalarda PC'ye aktarılır.

Çakışma (hazard) durumlarına karşı, **forwarding mekanizmaları** bu aşamada aktif olarak rol oynar. Eğer A ya da B operandları önceki aşamalarda değiştirilecekse, doğru veriler Hazard Detection Unit tarafından belirlenerek MUX'lar üzerinden yönlendirilir. Forward A ve Forward B sinyalleri yardımıyla, ALU'ya yazma aşamasında olan veriler

eksiksiz şekilde ulaştırılır. Böylece veri tutarlılığı sağlanır ve pipeline akışı kesintiye uğramadan devam eder.

Son olarak, bu aşamada elde edilen **ALU sonucu**, **branch hedef adresi**, bazı kontrol sinyalleri ve belirli veri alanları bir sonraki aşama olan MEM'e taşınmak üzere EX/MEM register'larına aktarılır. Bu register'lar, her clock çevriminde bu bilgileri yakalayarak sistemin senkronize şekilde çalışmasını sağlar.

Memory Access (MEM) aşaması, özellikle Load ve Store komutları için işlemcinin RAM (veri belleği) ile doğrudan etkileşime geçtiği kritik aşamadır. Bu aşamada temel olarak RAM bileşeni, kontrol sinyalleri, adres ve veri yolları, bir multiplexer (MUX) ve sonuç verisini tutan bir data register kullanılır. Önceki Execute aşamasında ALU tarafından hesaplanan adres, RAM'in adres girişine iletilir. Aynı anda, register dosyasından elde edilen ve yazılması gereken veri de RAM'in veri girişine bağlanır. RAM üç temel sinyalle çalışır: adres girişi (ALU çıkışı), veri girişi (register'dan gelen veri) ve kontrol sinyalleri olan MemRead ve MemWrite. MemRead sinyali aktif olduğunda, verilen adresteki veri RAM'den okunur. MemWrite aktifse, ilgili adrese veri yazılır. Eğer her iki sinyal de pasifse, bellek erişimi yapılmaz; bu durumda doğrudan ALU sonucu kullanılır.

RAM çıkışının ardından bir MUX devreye girer. Bu MUX'un görevi, veri yolu üzerinden hangi değerin aktarılacağını belirlemektir: ya RAM'den gelen veri ya da doğrudan ALU sonucu. Eğer Load işlemi yapılmaktaysa MUX, RAM çıkışını seçer; aksi hâlde, ALU sonucu doğrudan seçilerek sistemin veri akışı sürdürülür. Bu seçimin kontrolü MemToReg sinyaliyle Control Unit tarafından sağlanır. MUX'un çıkışı, verinin geçici olarak tutulduğu bir data register'a bağlanır. Bu register, Write Back aşamasına geçişte veriyi hazır hâlde tutar. Aynı zamanda, bu aşamada aktif olan hedef register adresi (rd3) de belirlenir. Böylece hangi register'a yazma yapılacağı netleşmiş olur. Ek olarak, EX/MEM register'ında tutulan kontrol sinyalleri bu aşamada MEM/WB register'ına aktarılır ve sonraki aşama için hazırlanır.

Write Back (WB) aşaması, komutun çıktısının işlemcinin Register File'ına yazıldığı ve sistemde kalıcı hâle getirildiği son aşamadır. Bu aşamada iki farklı veri kaynağı mevcuttur: RAM'den gelen veri ya da ALU sonucu. Hangisinin yazılacağı, MemToReg sinyali tarafından kontrol edilen bir MUX ile belirlenir. Eğer MemToReg sinyali 1 ise, yazma işlemi RAM'den gelen veriyle yapılır (örneğin Load komutlarında); eğer sinyal 0 ise, yazma işlemi ALU çıktısı kullanılarak gerçekleştirilir (örneğin aritmetik işlemler).

Seçilen veri, daha önce MEM aşamasında belirlenmiş olan hedef register adresine (rd3) yazılır. Yazma işleminin gerçekleştirilip gerçekleştirilmeyeceği RegWrite kontrol sinyali ile belirlenir. Eğer RegWrite sinyali aktifse (1), veri Register File'a yazılır; pasifse (0), herhangi bir yazma işlemi yapılmaz. Bu aşamada kullanılan tüm kontrol sinyalleri, MEM aşamasında MEM/WB register'ına aktarılmış olduğundan, sistemin pipeline boyunca

tutarlı şekilde ilerlemesi sağlanır. Ayrıca bu aşamada üretilen sonuç verisi, eğer bir sonraki komutta kullanılacaksa Forwarding mekanizması aracılığıyla ALU'ya veya başka MUX'lara yönlendirilerek çakışmaların önüne geçilir.

Stall, Bubble ve Kill Mekanizmaları

Pipelined mimarilerde veri ve kontrol tehlikelerinin yönetilmesi için çeşitli durdurma (stall) ve geçici boş komut ekleme (bubble) mekanizmaları gereklidir. Projemizde bu mekanizmalar, **Hazard Detection Unit**, **Control Unit**, ve **PC Control** gibi birimlerin koordinasyonuyla uygulanmıştır.

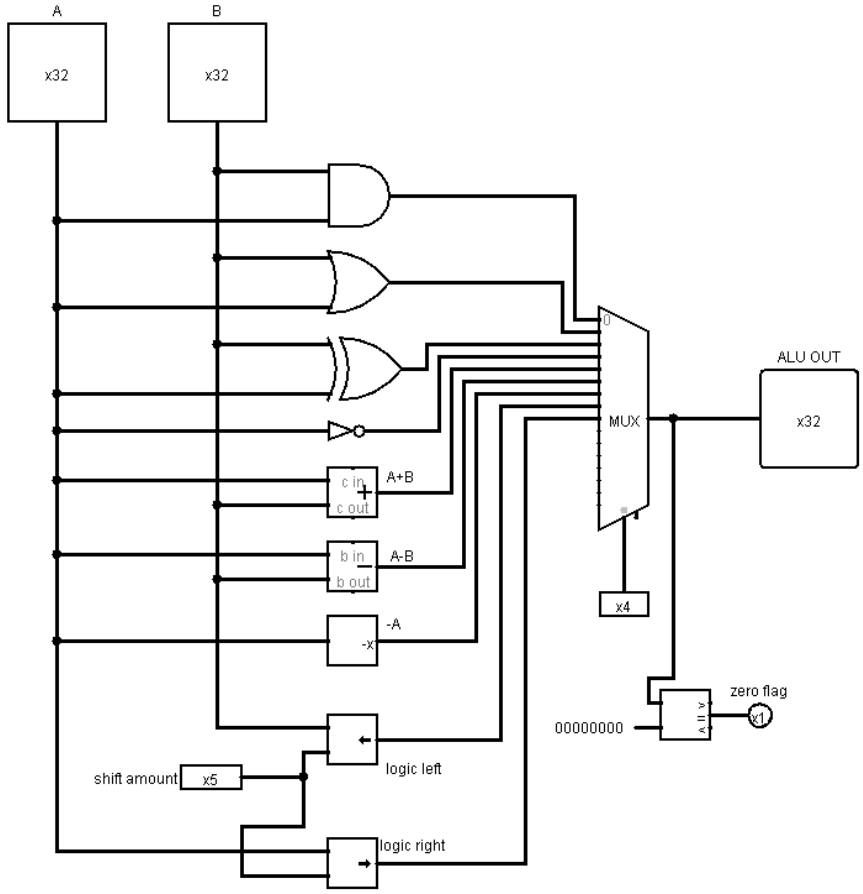
Öncelikle, eğer yürütülmekte olan bir komut belleğe yazma yapacaksa ve onu takip eden komut aynı register'dan veri okumaya çalışıyorsa, bu durumda veri çakışması (data hazard) meydana gelir. Bu tür durumlarda sistemin bir döngü beklemesi gerekir. Bu amaçla **Hazard Detection Unit**, stall sinyali üretir. stall sinyali, iki farklı bileşeni kontrol eder: Birincisi, Control Unit çıkışına yerleştirilmiş bir MUX; bu MUX, stall sinyali aktif olduğunda kontrol sinyallerini 0 yaparak "bubble" üretir. İkincisi ise, IF aşamasındaki register'ların enable (etkinleştirme) girişleridir. stall sinyali aktifken, **Program Counter (PC)**, **Instruction Register (IR)** ve **Next PC Register (NPC)** gibi IF aşamasına ait register'ların enable girişleri sıfırlanır ve sistem yeni bir komut almaz. Böylece mevcut komut bir döngü bekletilir, ve bu sayede pipeline içerisine bir bubble yerleştirilmiş olur.Bir diğer önemli mekanizma ise **branch komutları**yla ilgilidir. Eğer bir branch komutu yürütülecekse ve dallanma kararı alınacaksa, IF aşamasında alınan sonraki komutlar geçersiz hâle gelebilir. Bu durumlarda **bubble yerleştirmek** ve yanlış alınmış komutları etkisiz hale getirmek gerekir. Bu işlev için projede **iki ayrı "kill" sinyali** tanımlanmıştır: kill1 ve kill2.Kill1 sinyali, **Instruction Memory** çıkışı ile **Instruction Register** girişi arasındaki MUX'a bağlıdır. Bu sinyal aktif olduğunda MUX, instruction yerine sıfır (NOP) geçirerek bir bubble oluşturur. Kill2 sinyali ise, **Control Unit** çıkışından sonra yer alan başka bir MUX'a bağlanır. Bu MUX da kontrol sinyallerini sıfıra çekerek komutun sistemde ilerlemesini engeller.Her iki kill sinyali, stall sinyali ve branch durumu birleştirilerek bir OR kapısından geçirilir. Böylece sistem hem veri tehlikesi hem de kontrol tehlikesi tespit ettiğinde doğru şekilde bubble üretir. Bu yapı sayesinde pipeline'ın istikrarı korunmuş olur.

Hex Digit Display

Sistemde test işlemlerini kolaylaştırmak ve komut akışının hangi aşamada olduğunu görselleştirmek için üst kısma bir **HexDigitDisplay** bileşeni yerleştirilmiştir. Bu gösterge, örneğin A1 register'ının değerini takip etmekte ve komut sıralaması boyunca A10'dan 0'a kadar gerçekleşen azalma işlemlerini izlemeyi sağlar. Böylece sistemin test edilebilirliği ve izlenebilirliği artırılmıştır.

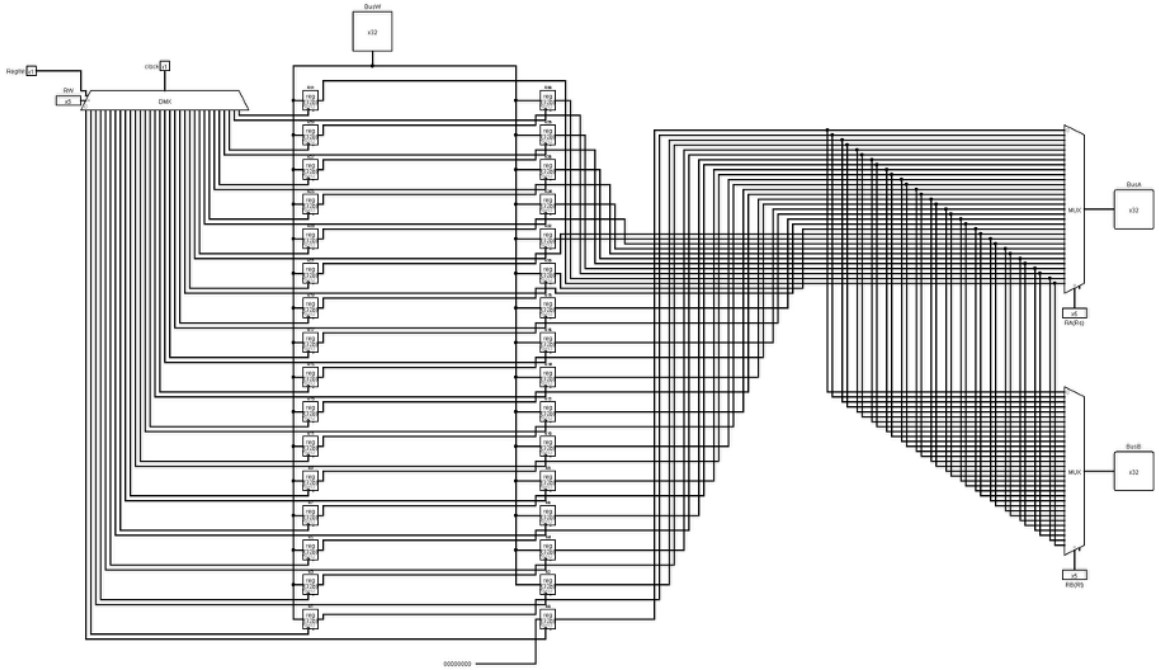
Alu

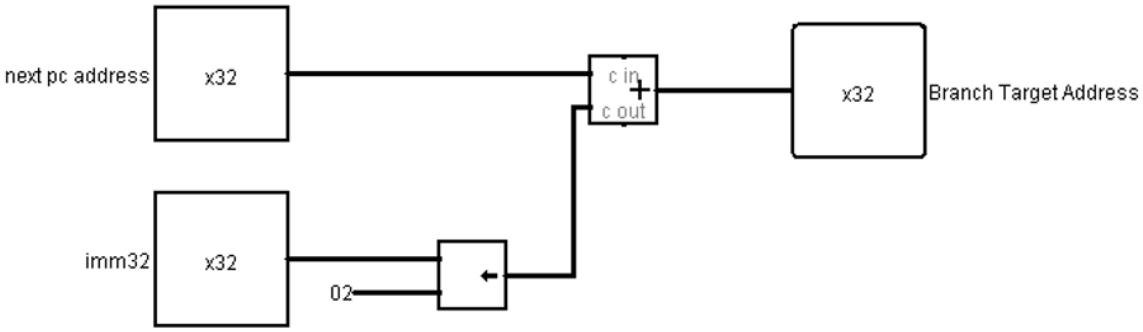
Alu temel mantıksal işlemleri, temel aritmetik işlemleri ve kullanacağımız mantıksal öteleme işlemini yapmaktadır. İki adet 32 bit girişi, bir ader shift amount(sa) girişi, kontrol girişleri, 32 bitlik bir çıkışı ve zero flag çıkışı vardır. Öteleme işlemleri için aritmetik library içerisinden shifter kullandık. ALU'da yapılan bütün işlemler bir multiplexer'a bağlanıyor. Multiplexer'ın 4 bitlik seçme girişi kontrol ünitesi tarafından belirleniyor. Eğer MUX'un seçme girişi 0 olursa and işlemi, 1 olursa or işlemi, 2 olursa xor, 3 olursa not, 4 -> toplama, 5 -> çıkarma, 6 -> negatif yapma, 7 -> mantıksal sola öteleme, 8 -> mantıksal sağa öteleme işlemleri yapılıyor. Eğer işlem sonucu sıfır çıkarsa zero flag=1, sıfırdan farklı bir değer çıkarsa zero flag=0 yapılıyor.



Registers

Registers bloğunda 32 adet 32 bitlik register bulunur. İlk register R0 sabit ve her zaman sıfır değerlidir. Hem registerlara yazma işlemi hemde registerdan okuma işlemi yapılır. Register clock atımları bir demultiplexer'a bağlıdır. DEMUX'un seçme girişi RW tarafından alınır ve hangi registera yazma işlemi yapılacağı belirlenir. Hangi register seçilmişse o register'ın clock atımı aktifleşir ve BusW girişindeki değer register'a yüklenir. Her register'ın çıkışı da iki adet multiplexer'a bağlanır. Bunlardan biri BusA'yı öteki BusB'yi temsil etmektedir. BusA MUX'unun seçme girişi RA ve BusB MUX'unun seçme girişi RB'dir. Okunmak istenen register'ın adresi RA ve RB ile girilir, böylece o adresteki register değeri Bus'a iletilmiş olur.



+ Bloğu

İki adet 32 bit girişi ve bir adet 32 bit çıkışı vardır. İnstruction register'ındaki 16 bit immediate değeri extender ile 32 bite yükseltilir. Ardından 32 bite yükseltilmiş immediate değeri bloğun girişine bağlanır. Bu değer bloğun içerisinde logic shifter kullanılarak 4 ile çarpılır ve bir sonraki PC adres (NPC) ile toplanır. Sonuç 32 bit olarak BTA (branch target address) register'a yazılır.